

ML Lab Week 14: CNN Image Classification

1. Introduction

The objective of this lab exercise is to design, implement, train, and evaluate a Convolutional Neural Network (CNN) model using PyTorch to classify images of hand gestures into one of three categories: rock, paper, or scissors. The model is trained on a labeled image dataset and its performance is assessed on unseen test data.

2. Model Architecture

The CNN model consists of three convolutional blocks, each followed by a ReLU activation function and MaxPooling. The input images are resized to 128x128 with three color channels.

Convolutional Layers:

- Conv2D: $3 \rightarrow 16$ channels, kernel size 3, padding 1, followed by ReLU and MaxPool2D(2)
- Conv2D: $16 \rightarrow 32$ channels, kernel size 3, padding 1, followed by ReLU and MaxPool2D(2)
- Conv2D: $32 \rightarrow 64$ channels, kernel size 3, padding 1, followed by ReLU and MaxPool2D(2)

After three pooling layers, the spatial resolution is reduced from $128 \rightarrow 64 \rightarrow 32 \rightarrow 16$, resulting in a flattened feature vector of size $64 \times 16 \times 16$.

Fully Connected Classifier:

- Flatten
- Linear layer: $64 \times 16 \times 16 \rightarrow 256$
- ReLU activation
- Dropout ($p = 0.3$)
- Final Linear layer: $256 \rightarrow 3$ (representing three class outputs)

3. Training and Performance

The model was trained on 80 percent of the dataset, while the remaining 20 percent was used for testing.

Training Hyperparameters:

- Optimizer: Adam
- Loss Function: CrossEntropyLoss
- Learning Rate: 0.001

- Batch Size: 32
- Epochs: 10

Final Test Accuracy Achieved: 99.32%

4. Conclusion and Analysis

The model demonstrated excellent performance, achieving a test accuracy of 99.32%. This indicates a strong capability of the CNN in distinguishing between rock, paper, and scissors gestures. The training process was stable and no major issues were encountered. Minor challenges involved tuning hyperparameters for optimal results.

Potential improvements to further enhance performance could include:

- Incorporating data augmentation to improve generalization.
- Increasing the number of training epochs.
- Adding batch normalization or additional dropout layers to reduce overfitting.

The model successfully meets the objectives of the lab and performs highly effectively on the classification task.